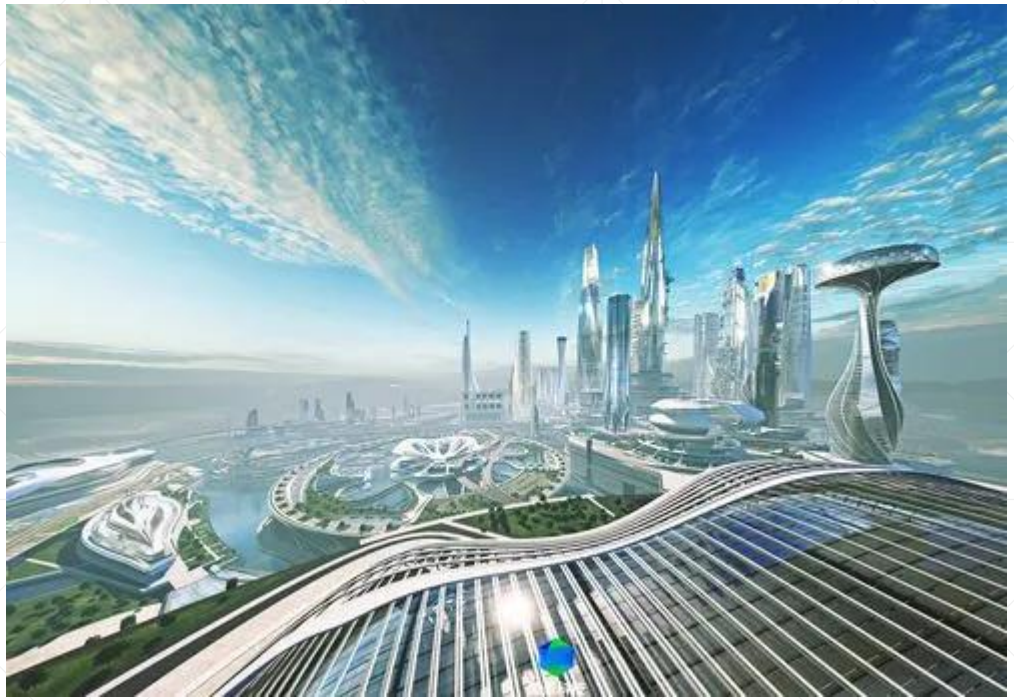
使用CompletableFuture之三



异步任务间的配合

北京理工大学计算机学院
金旭亮

等待多个任务的完成-1

Cooperation.java

```
private static void useAllOfToWait() {
    var first :CompletableFuture<Integer> = CompletableFuture.supplyAsync(() -> {
        System.out.println("执行异步任务一");
        return 1;
    });
    var second :CompletableFuture<Integer> = CompletableFuture.supplyAsync(() -> {
        System.out.println("执行异步任务二");
        return 2;
    });
    var third :CompletableFuture<Integer> = CompletableFuture.supplyAsync(() -> {
        System.out.println("执行异步任务三");
        return 3;
    });
```

待续……

等待多个任务的完成-2

接上页……

```
var all :CompletableFuture<Void> = CompletableFuture.allOf(first, second, third);
all.thenRun(() -> {
    try {
        System.out.println("所有异步任务都已经执行完毕了！");
        var finalResult = first.get() + second.get() + third.get();
        System.out.println("结果为: " + finalResult);
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();
    }
});
```

等待最先完成的线程给出的结果

```
private static void useAnyOfToWaitFastest() {
    Supplier<String> stringSupplier = () -> {
        int ranValue = new Random().nextInt( bound: 10000);
        var threadName :String = Thread.currentThread().getName();
        var info = threadName + "休眠" + ranValue + "毫秒";
        System.out.println(info);
        try {
            Thread.sleep(ranValue);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        return info;
    };
    var first :CompletableFuture<String> = CompletableFuture.supplyAsync(stringSupplier);
    var second :CompletableFuture<String> = CompletableFuture.supplyAsync(stringSupplier);
    CompletableFuture.anyOf(first, second).thenAccept(result -> {
        System.out.println("收到最快的线程传出的信息为: " + result);
    }).join();
}
```

```
Run:  CompletableFuturesDemo
"C:\Program Files\Java\jdk-17\bin\java.exe" --module-path "C:\Program
ForkJoinPool.commonPool-worker-2休眠9983毫秒
ForkJoinPool.commonPool-worker-1休眠3184毫秒
收到最快的线程传出的信息为: ForkJoinPool.commonPool-worker-1休眠3184毫秒

Process finished with exit code 0
```

限时等待

```java
private static void timeOutDemo() {
    var future :CompletableFuture<String> = CompletableFuture.supplyAsync(() -> {
        try {
            Thread.sleep( millis: 3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        return "finished";
    });
    try {
        //最长等1秒钟，超时后抛出TimeoutException
        var result :String = future.orTimeout( timeout: 1, TimeUnit.SECONDS)
                .get();
        System.out.println(result);
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();
    }
}
```

Timeout.java

超时后指定默认值

```
private static void timeOutDemo2() {
    var future :CompletableFuture<String> = CompletableFuture.supplyAsync(() -> {
        try {
            Thread.sleep( millis: 3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        return "finished";
    });
    try {
        //最长等1秒钟，超时后给一个默认值
        var result :String = future.completeOnTimeout( value: "超时之后的默认值",
                timeout: 1, TimeUnit.SECONDS).get();
        System.out.println(result);
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();
    }
}
```

默认值通过参数指定